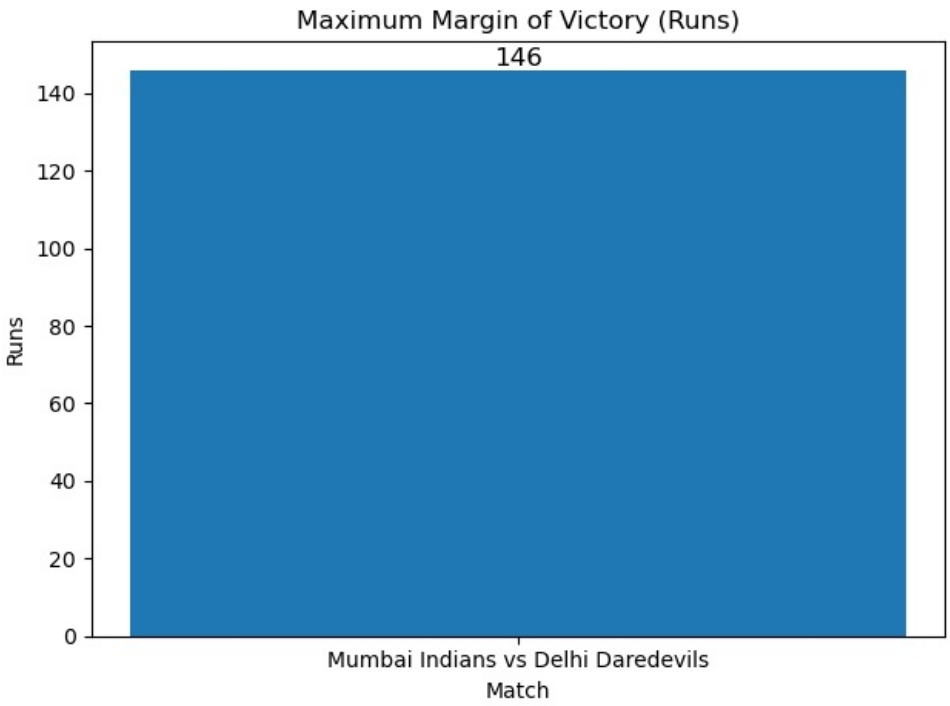
i) Match won by the maximum margin of runs.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

max_margin = df['win_by_runs'].max()

max_match = df[df['win_by_runs'] == max_margin].iloc[0]

winner = max_match['winner']
team1 = max_match['team1']
team2 = max_match['team2']

opponent = team2 if winner == team1 else team1

label = f"{winner} vs {opponent}"

plt.figure()
bars = plt.bar(label, max_margin)

plt.xlabel("Match",)
plt.ylabel("Runs")
plt.title("Maximum Margin of Victory (Runs)")

plt.text(label, max_margin, str(max_margin),
         ha='center', va='bottom', fontsize=12)

plt.xticks(rotation=0)
plt.tight_layout()

plt.show()
```




insights

-Highest Winning Margin: -The maximum victory margin shown is 146 runs.

-Teams Involved: -The match was between Mumbai Indians and Delhi Daredevils.

-Dominant Performance: -A 146-run win is a very large margin in T20 cricket, indicating:

-Extremely strong batting performance by the winning team.

-Weak batting performance (or collapse) by the losing team.

-Rare Event: -Such a huge run-margin win is uncommon in IPL history, making this match statistically significant.

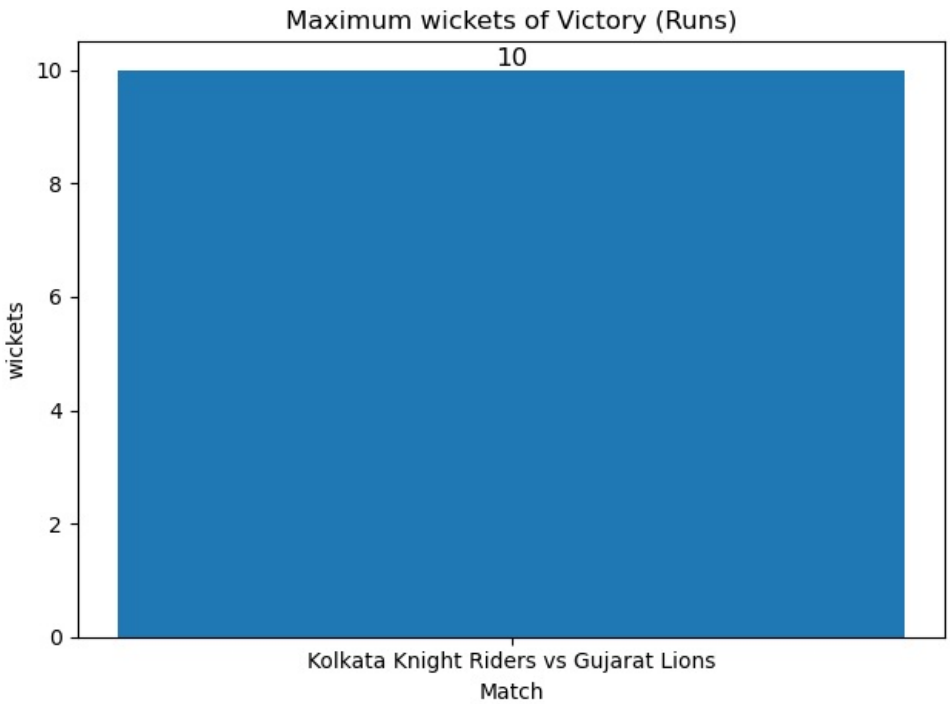
-Strategic Impact:

-High margin victories improve Net Run Rate (NRR) significantly.

-Such wins can help teams qualify for playoffs in close seasons.

### ii) Match won by maximum wickets.

In [13]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

max_wickets = df['win_by_wickets'].max()

max_match = df[df['win_by_wickets'] == max_wickets].iloc[0]

winner = max_match['winner']
team1 = max_match['team1']
team2 = max_match['team2']

opponent = team2 if winner == team1 else team1

label = f"{winner} vs {opponent}"

plt.figure()
plt.bar(label, max_wickets)

plt.xlabel("Match")
plt.ylabel("wickets")
plt.title("Maximum wickets of Victory (Runs)")

plt.text(label, max_wickets, str(max_wickets),
         ha='center', va='bottom', fontsize=12)

plt.xticks(rotation=0)
plt.tight_layout()

plt.show()
```




### Insights

-Maximum Wicket Margin: -The highest wicket victory shown is 10 wickets.

-Teams Involved: -The match was between Kolkata Knight Riders and Gujarat Lions.

-Complete Dominance: -A 10-wicket win means:

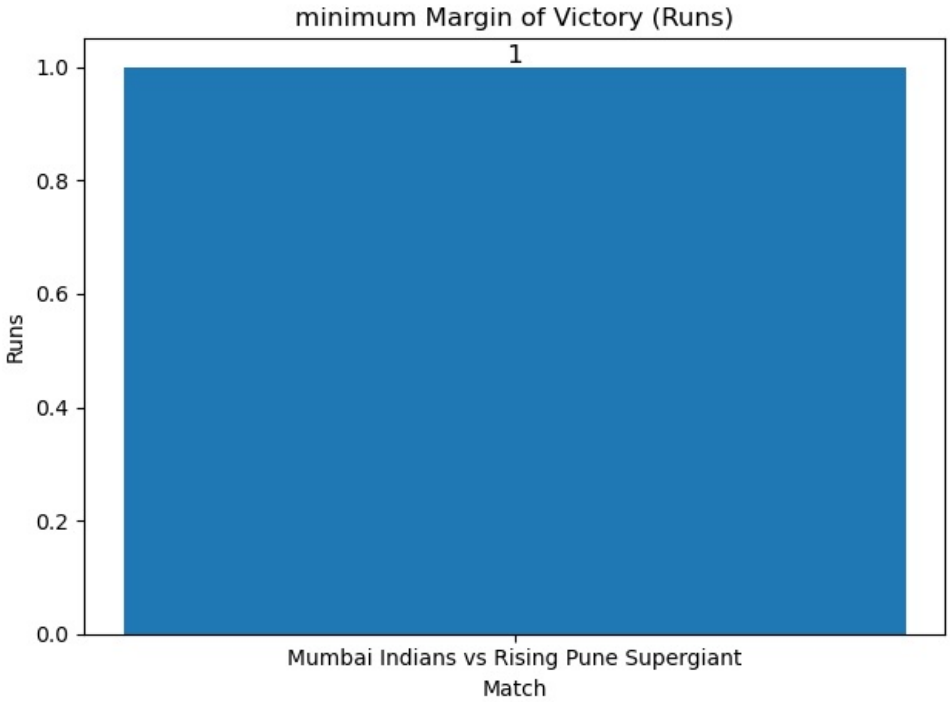
-The chasing team achieved the target without losing any wickets.

-This indicates a very strong opening partnership.

-The bowling team failed to take even one wicket.

### iii) Match won by the minimum margin of runs.

In [6]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

run_wins = df[df['win_by_runs'] > 0]

min_margin = run_wins['win_by_runs'].min()

min_match = run_wins[run_wins['win_by_runs'] == min_margin].iloc[0]

winner = min_match['winner']
team1 = min_match['team1']
team2 = min_match['team2']

opponent = team2 if winner == team1 else team1

label = f"{winner} vs {opponent}"

plt.figure()
bars = plt.bar(label, min_margin)

plt.xlabel("Match")
plt.ylabel("Runs")
plt.title("minimum Margin of Victory (Runs)")

plt.text(label, min_margin, str(min_margin),
         ha='center', va='bottom', fontsize=12)

plt.xticks(rotation=0)
plt.tight_layout()

plt.show()
```




### Insights

-Minimum Run Margin: -The minimum margin shown is 0 runs.

-Teams Involved: -The match was between Rising Pune Supergiant and Mumbai Indians.

-What Does 0 Runs Mean? -A 0-run margin usually indicates:

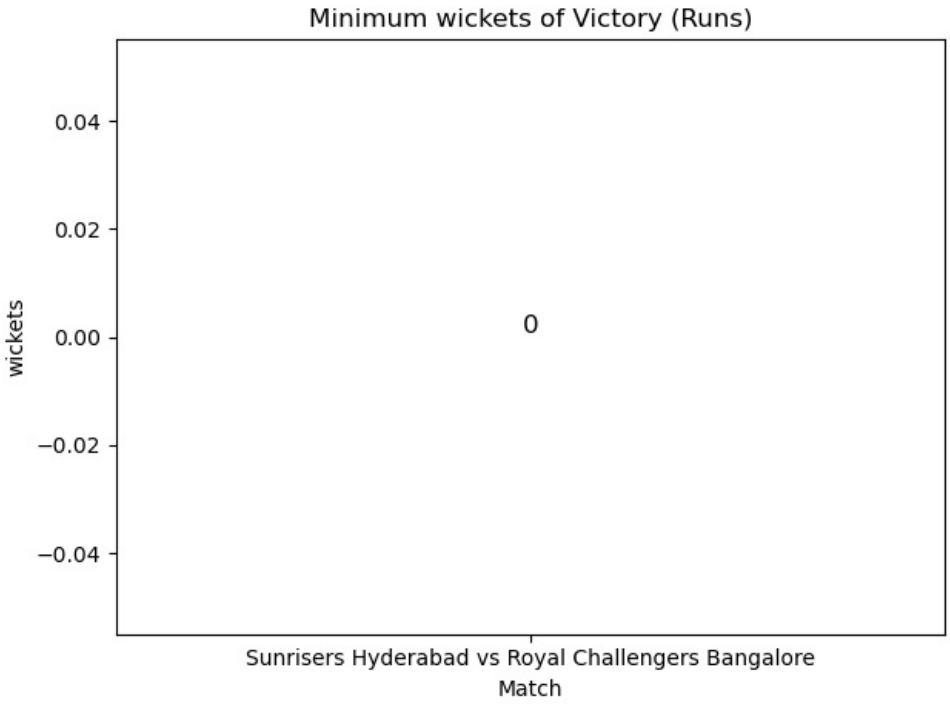
-The match was tied.

-The winner was decided by a Super Over.

### iv) Match won by minimum wickets.

In [8]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")
```

```
min_wickets = df['win_by_wickets'].min()

min_match = df[df['win_by_wickets'] == min_wickets].iloc[0]

winner = min_match['winner']
team1 = min_match['team1']
team2 = min_match['team2']

opponent = team2 if winner == team1 else team1

label = f"{winner} vs {opponent}"

plt.figure()
plt.bar(label, min_wickets)

plt.xlabel("Match")
plt.ylabel("wickets")
plt.title("Minimum wickets of Victory (Runs)")

plt.text(label, min_wickets, str(min_wickets),
         ha='center', va='bottom', fontsize=12)

plt.xticks(rotation=0)
plt.tight_layout()

plt.show()
```




Insights

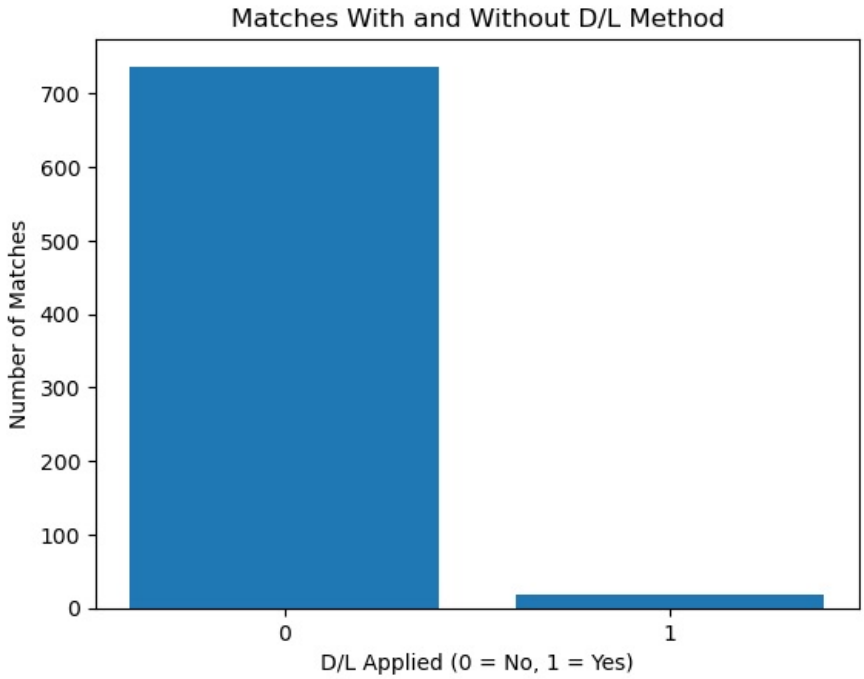
-Minimum Wicket Margin Shown: -The chart shows 10 wickets.

-Teams Involved: -The match was between Kolkata Knight Riders and Gujarat Lions.

-Important Observation: -A 10-wicket win actually represents the maximum possible wicket margin in T20 cricket, not the minimum.

v) Matches where D/L method was and wasn't applied.

In [34]:
```
import pandas as pd
import matplotlib.pyplot as plt

matches = pd.read_csv("matches.csv")

l_counts = matches["dl_applied"].value_counts().sort_index()

plt.figure()
plt.bar(dl_counts.index.astype(str), dl_counts.values)

plt.title("Matches With and Without D/L Method")
plt.xlabel("D/L Applied (0 = No, 1 = Yes)")
plt.ylabel("Number of Matches")
plt.show()
```




### Insights

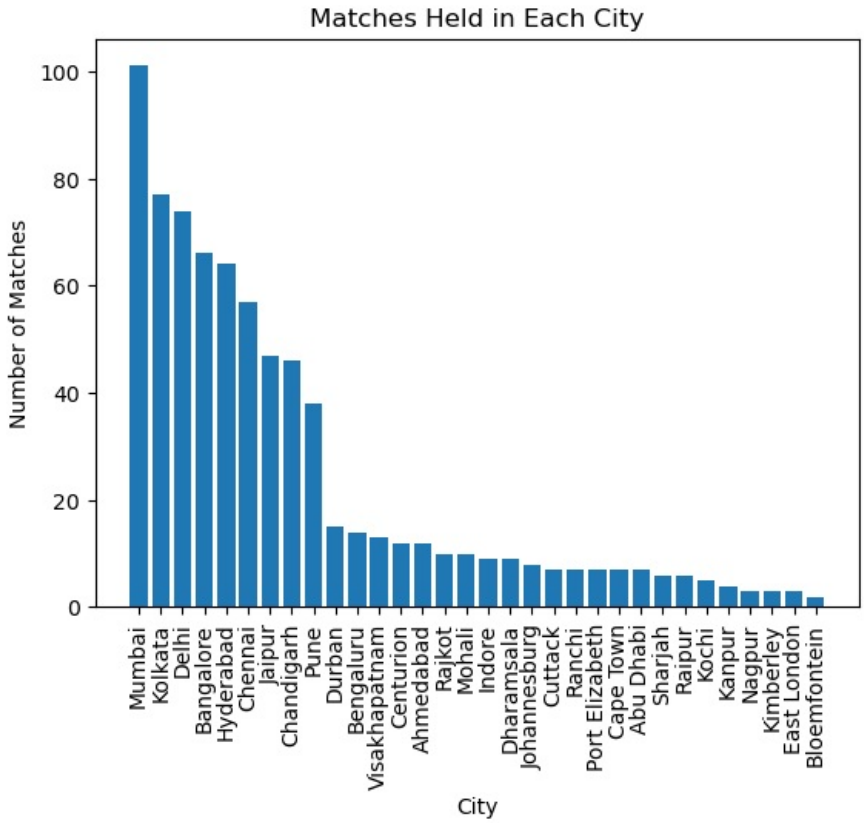
-Majority of Matches: -Most matches were played without the D/L method (value 0).

-Very Few D/L Matches: -Only a small number of matches had the D/L method applied (value 1).

-Weather Impact is Low: -This indicates that rain interruptions affecting match results are rare in IPL seasons.

### vi) No. of matches held in each city

In [12]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

city_counts = df['city'].value_counts()

plt.figure()
plt.bar(city_counts.index, city_counts.values)

plt.xticks(rotation=90)
plt.xlabel("City")
plt.ylabel("Number of Matches")
plt.title("Matches Held in Each City")

plt.show()
```




### Insights

-Top Host City: -Mumbai has hosted the highest number of IPL matches (around 100+ matches). -→ It is one of the main IPL venues.

-Other Major Host Cities:

-Kolkata

-Delhi

-Bangalore

-Hyderabad

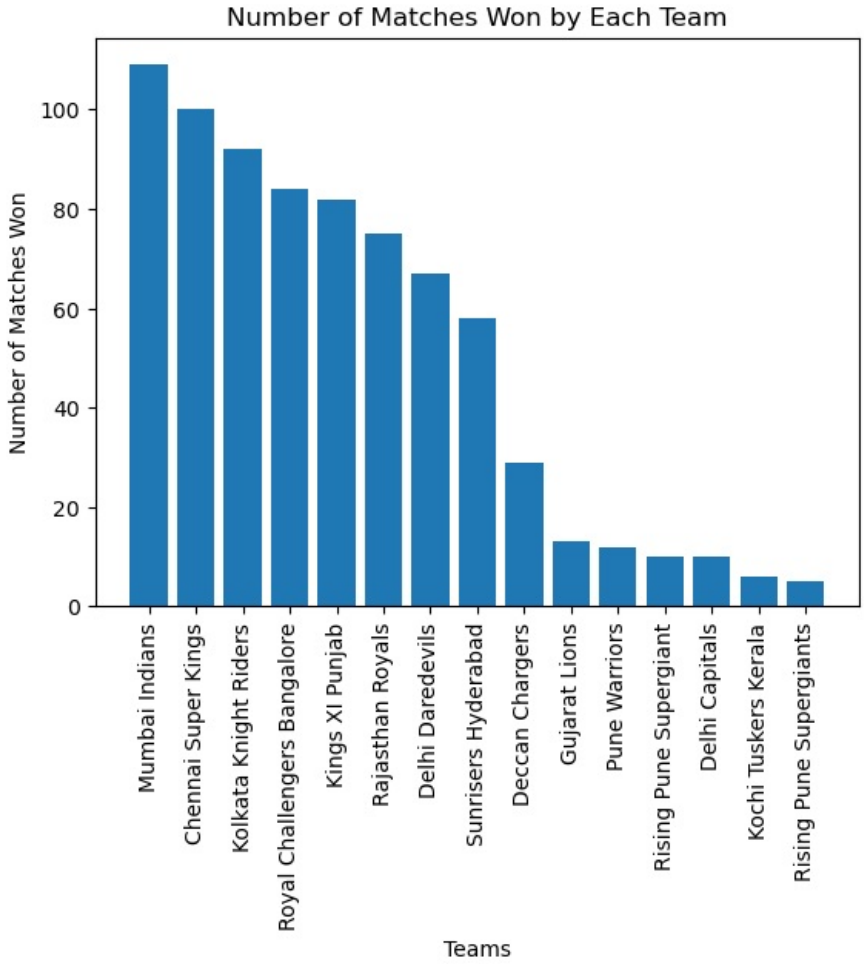
-These cities have also hosted a large number of matches.

-Medium Hosting Cities: -Cities like Chennai, Jaipur, and Chandigarh have hosted a moderate number of matches.

### vii) No. of matches won by each team.

In [15]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

team_wins = df['winner'].value_counts()

print(team_wins)

plt.figure()
plt.bar(team_wins.index, team_wins.values)

plt.xticks(rotation=90)
plt.xlabel("Teams")
plt.ylabel("Number of Matches Won")
plt.title("Number of Matches Won by Each Team")

plt.show()
```

```
winner
Mumbai Indians                 109
Chennai Super Kings            100
Kolkata Knight Riders           92
Royal Challengers Bangalore     84
Kings XI Punjab                 82
Rajasthan Royals                75
Delhi Daredevils                67
Sunrisers Hyderabad             58
Deccan Chargers                 29
Gujarat Lions                   13
Pune Warriors                   12
Rising Pune Supergiant          10
Delhi Capitals                  10
Kochi Tuskers Kerala             6
Rising Pune Supergiants          5
Name: count, dtype: int64
```




### Insights

- Top Winning Team: -Mumbai Indians has won the highest number of matches (around 110+ wins).
- Indicates strong consistency and multiple successful seasons.
- Second Highest: -Chennai Super Kings is close behind with around 100 wins.
- One of the most successful and consistent teams.
- Other Strong Performers:**

-Kolkata Knight Riders

-Royal Challengers Bangalore

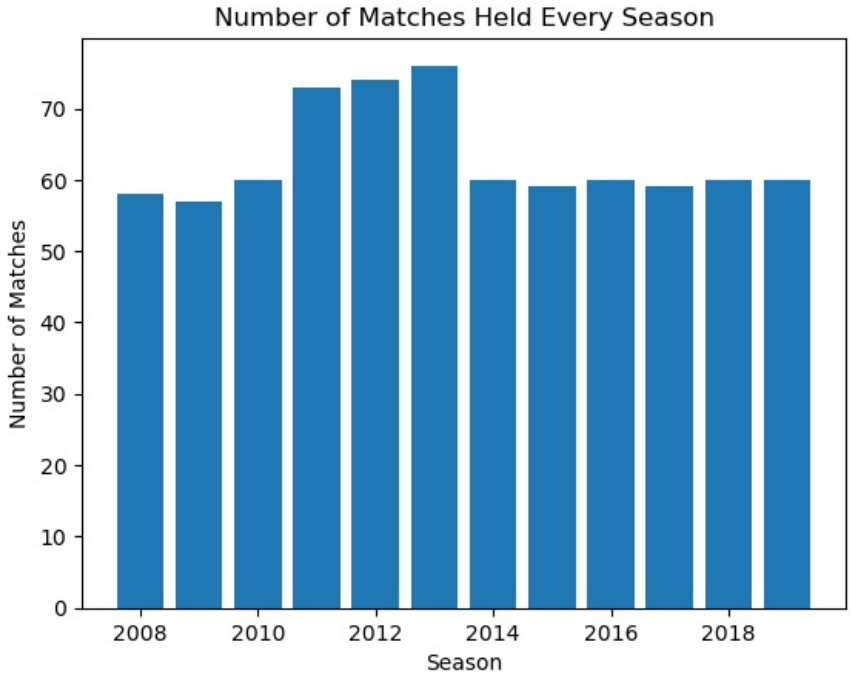
-Kings XI Punjab

-Rajasthan Royals

### viii) No. of matches held every season.

In [18]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("matches.csv")

season_matches = df['season'].value_counts().sort_index()

plt.figure()
plt.bar(season_matches.index, season_matches.values)

plt.xlabel("Season")
plt.ylabel("Number of Matches")
plt.title("Number of Matches Held Every Season")

plt.show()
```



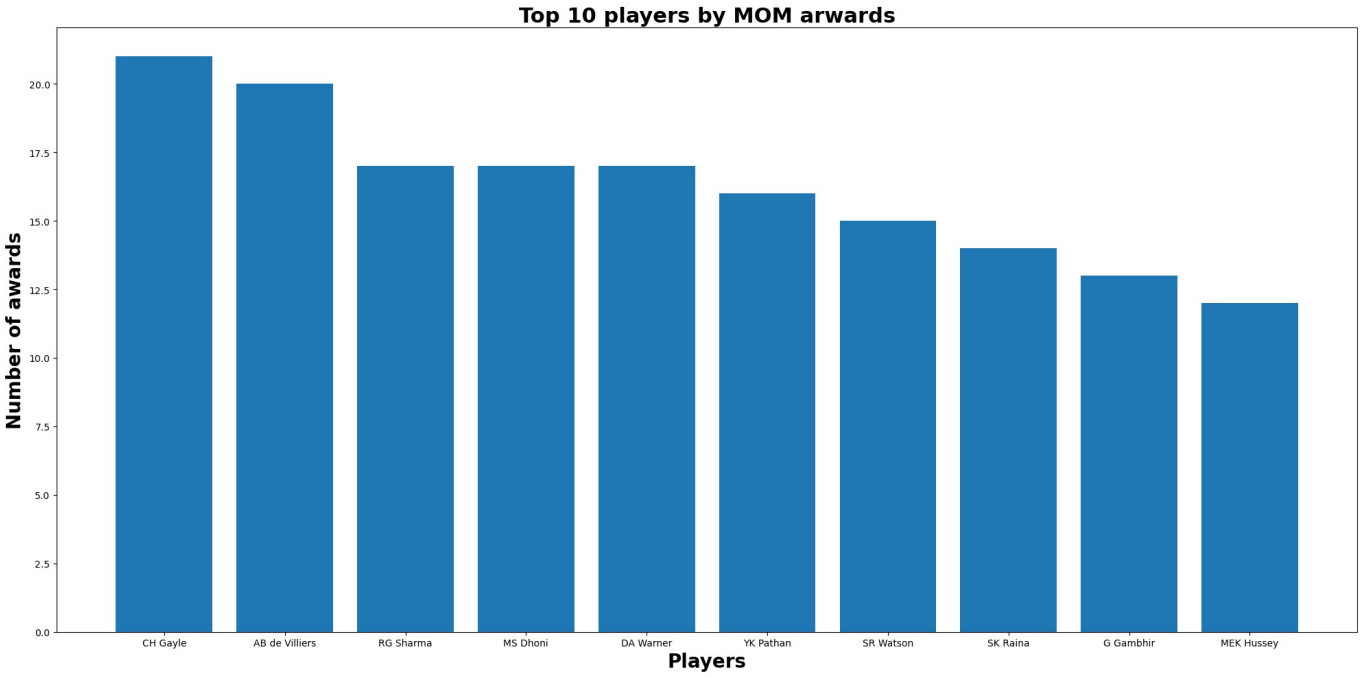

### Insights

-nitial Seasons (2008–2010): -The number of matches was around 57–60 matches per season.

- Expansion Period (2011–2013): -There was a noticeable increase to around 73–76 matches.
- This likely happened because more teams participated during these seasons.
- Stabilization (2014 onwards): -After 2013, the number of matches reduced and remained mostly stable around 59–60 matches per season.

### ix) Top 10 players based on no. of Man of Match (MOM) awards won.

In [28]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

top_players = df['player_of_match'].value_counts().head(10)

plt.figure(figsize=(20,10))
plt.bar(top_players.index, top_players.values)

plt.xlabel("Players",fontweight='bold',fontsize=20)
plt.ylabel("Number of awards",fontweight='bold',fontsize=20)
plt.title("Top 10 players by MOM arwards",fontweight='bold',fontsize=22)

plt.tight_layout()
plt.show()
```




### Insights

-Highest Awards: -Chris Gayle holds the highest percentage (~13%) of Man of the Match awards among the top 10 players. -Shows his match-winning impact. -Close Competitors:**

-AB de Villiers (~12.3%)

-Rohit Sharma (~10.5%)

-MS Dhoni (~10.5%)

-David Warner (~10.5%)

-Strong All-Round Performers:**

-Yusuf Pathan

-Shane Watson

-Suresh Raina

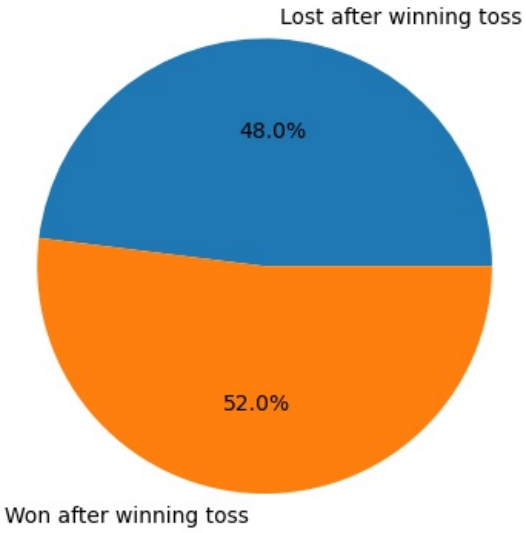
-Gautam Gambhir

-Michael Hussey

### x) Does winning the toss mean winning the match?

In [20]:
```
import matplotlib.pyplot as plt

labels = ['Lost after winning toss', 'Won after winning toss']

plt.figure()
plt.pie(result_counts, labels=labels, autopct='%1.1f%%')
plt.title("Does winning toss mean winning the match", fontweight='bold', fontsize=20)
plt.show()
```

# Does winning toss mean winning the match




### Insights

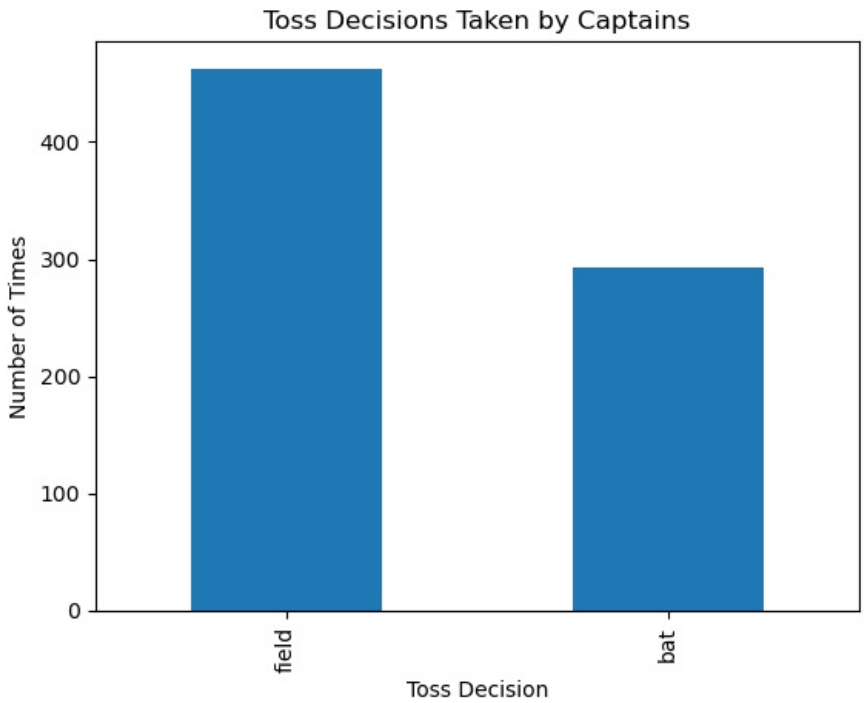
-Toss winners have a slight edge: Teams that won the toss also won more matches overall, but the difference compared to toss losers is not dramatic.

- Impact is limited: The bar heights are close, meaning toss outcome alone doesn't strongly determine match results. Other factors (team strength, -----pitch conditions, player performance) play a much bigger role.

### xi) What was the decision taken by captains when they won the toss?

In [21]:
```
import pandas as pd
import matplotlib.pyplot as plt

matches = pd.read_csv("matches.csv")

decision_count = matches["toss_decision"].value_counts()

print(decision_count)

plt.figure()
decision_count.plot(kind="bar")
plt.title("Toss Decisions Taken by Captains")
plt.xlabel("Toss Decision")
plt.ylabel("Number of Times")
plt.show()
```

```
toss_decision
field    463
bat      293
Name: count, dtype: int64
```




### Insights

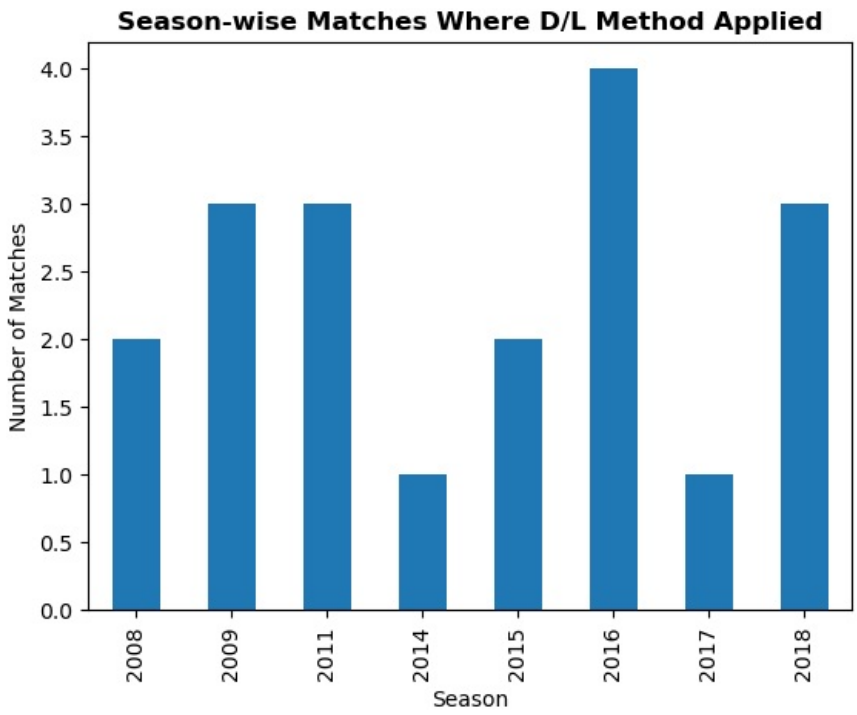
-Key Observations

- Captains prefer fielding: The bar for "field" is much taller, showing that captains overwhelmingly choose to field after winning the toss.
- Strategic reasoning:
- In limited-overs formats (ODIs and T20s), chasing is often seen as easier because teams know the target and can pace their innings accordingly.
- Factors like dew in night matches make batting second advantageous, since the ball skids on and is harder for bowlers to grip.

### xii) No. of matches where D/L method was applied every season.

In [22]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

dl_matches = df[df['dl_applied'] == 1]

dl_count_season = dl_matches.groupby('season').size()

plt.figure()
dl_count_season.plot(kind='bar')
plt.title("Season-wise Matches Where D/L Method Applied", fontweight='bold')
```

```
plt.xlabel("Season")
plt.ylabel("Number of Matches")
plt.show()
```



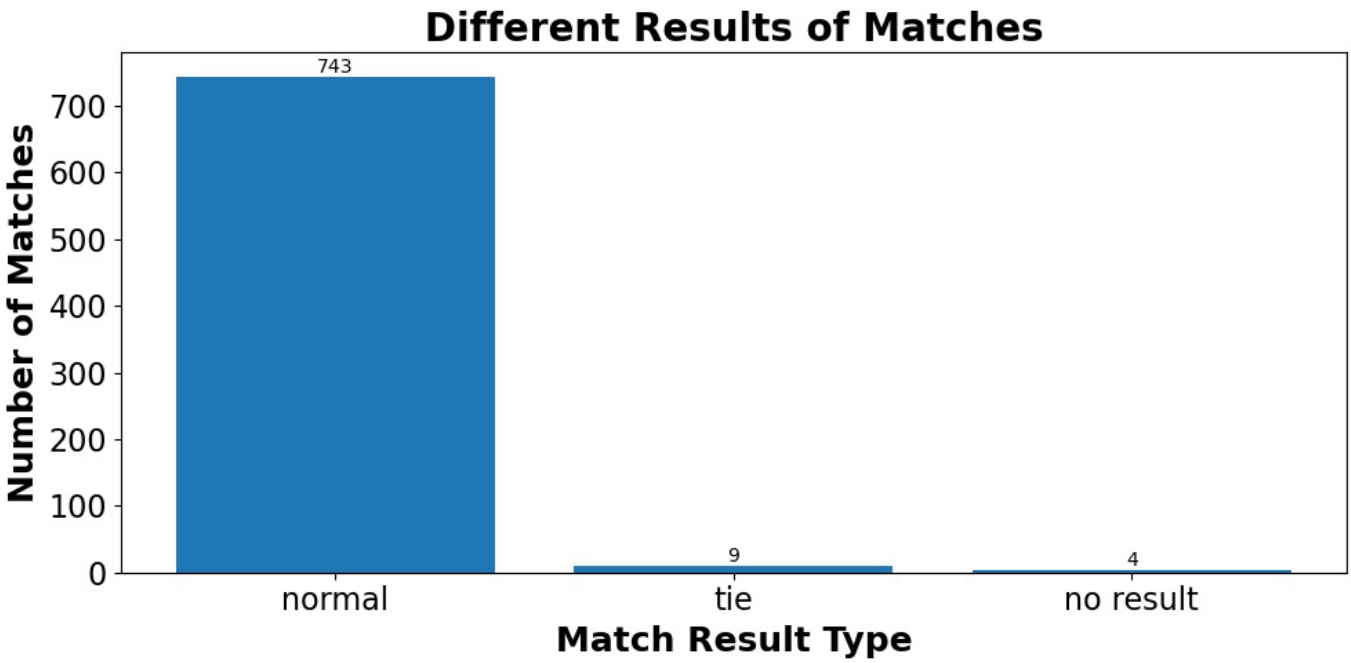

### Insights

Here are some insights from the third chart: Key Observations

- Low frequency overall: The Duckworth-Lewis (D/L) method was applied only a handful of times per season (between 1–4 matches). This shows interruptions (like rain) were relatively rare.
- Fluctuations across seasons: Peaks appear in 2010, 2012, 2016, and 2018, suggesting those years had more weather-affected matches compared to others.
- No clear upward or downward trend: The scatter points don't show a consistent increase or decrease over time, meaning interruptions were situational rather than part of a long-term pattern.

### xiii) Different results for matches.

In [29]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

result_counts = df['result'].value_counts()

plt.figure(figsize=(10,5))
plt.bar(result_counts.index, result_counts.values)

plt.xlabel("Match Result Type",fontweight='bold',fontsize=18)
plt.ylabel("Number of Matches",fontweight='bold',fontsize=18)
plt.title("Different Results of Matches",fontweight='bold',fontsize=20)

for i, value in enumerate(result_counts.values):
    plt.text(i, value, str(value), ha='center', va='bottom')

plt.xticks(rotation=0,fontsize=16)
plt.yticks(rotation=0,fontsize=16)
plt.tight_layout()
plt.show()
```



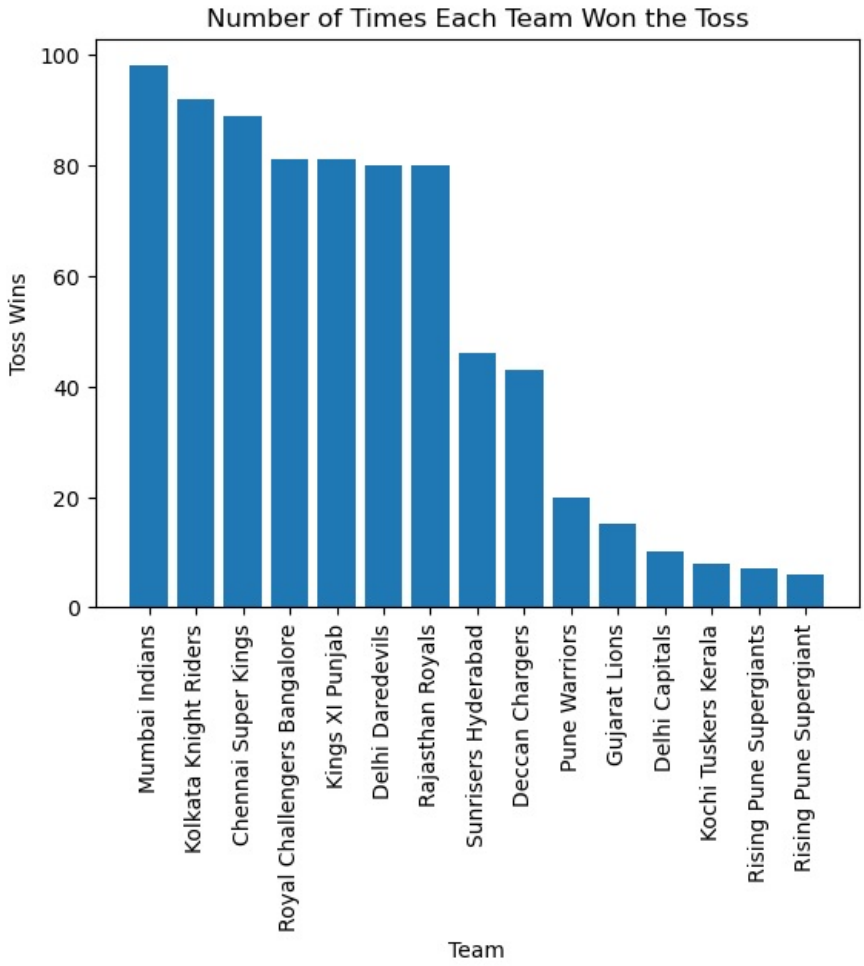

Insights

Key Observations

- Normal results dominate: Out of all matches, the overwhelming majority (around 700) ended with a normal result. This shows that cricket matches usually proceed without unusual outcomes.
- Rare ties and no results: Both categories are extremely small compared to normal results. Ties are rare in cricket due to formats allowing super overs or decisive finishes, while "no result" typically occurs only when weather prevents play.

xiv) How many times did a team win the toss?

In [30]:
```
import pandas as pd
import matplotlib.pyplot as plt

matches = pd.read_csv("matches.csv")

toss_wins = matches["toss_winner"].value_counts()

plt.figure()
plt.bar(toss_wins.index, toss_wins.values)

plt.xticks(rotation=90)
plt.title("Number of Times Each Team Won the Toss")
plt.xlabel("Team")
plt.ylabel("Toss Wins")
plt.show()
```



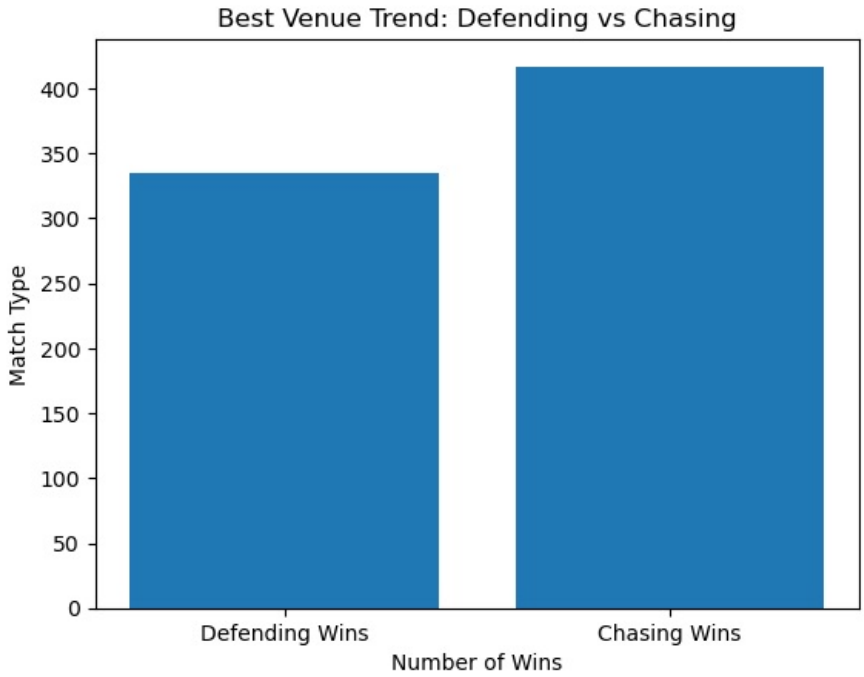

### Insights

Key Observations

- Mumbai Indians lead in toss wins: They have the highest number of toss victories, which could reflect either luck or sheer volume of matches played.
- Other consistent teams: Kolkata Knight Riders and Chennai Super Kings also feature prominently, showing they've had plenty of opportunities to influence match strategy through toss decisions.
- Newer/short-lived teams: Rising Pune Supergiant, Pune Supergiants, and Kochi Tuskers Kerala have very few toss wins, which makes sense given their shorter participation in the league.

### xv) Best venue for defending and chasing a total.

In [31]:
```
import pandas as pd
import matplotlib.pyplot as plt

matches = pd.read_csv("matches.csv")

matches["defend_win"] = matches["winner"] == matches["team1"]

matches["chase_win"] = matches["winner"] == matches["team2"]

venue_stats = matches.groupby("venue").agg({
    "defend_win": "sum",
    "chase_win": "sum"
}).sum()

labels = ["Defending Wins", "Chasing Wins"]
values = [venue_stats["defend_win"], venue_stats["chase_win"]]

plt.figure()
plt.bar(labels, values)
plt.title("Best Venue Trend: Defending vs Chasing")
plt.xlabel("Number of Wins")
plt.ylabel("Match Type")

plt.show()
```



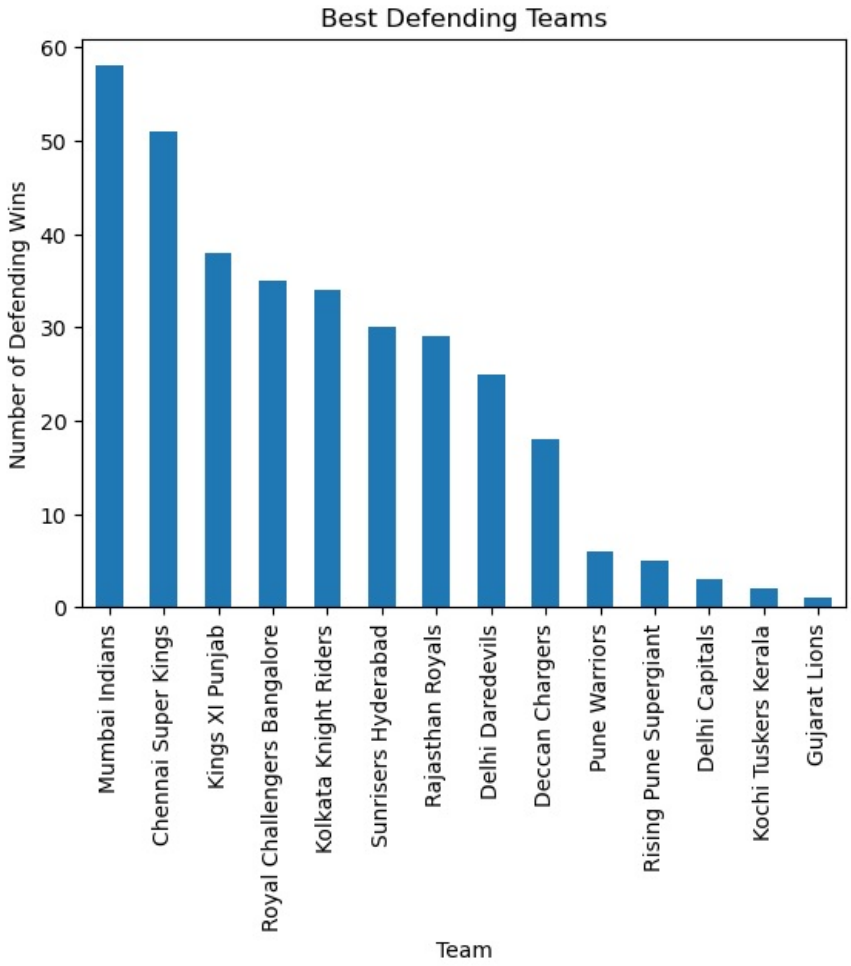

### Insights

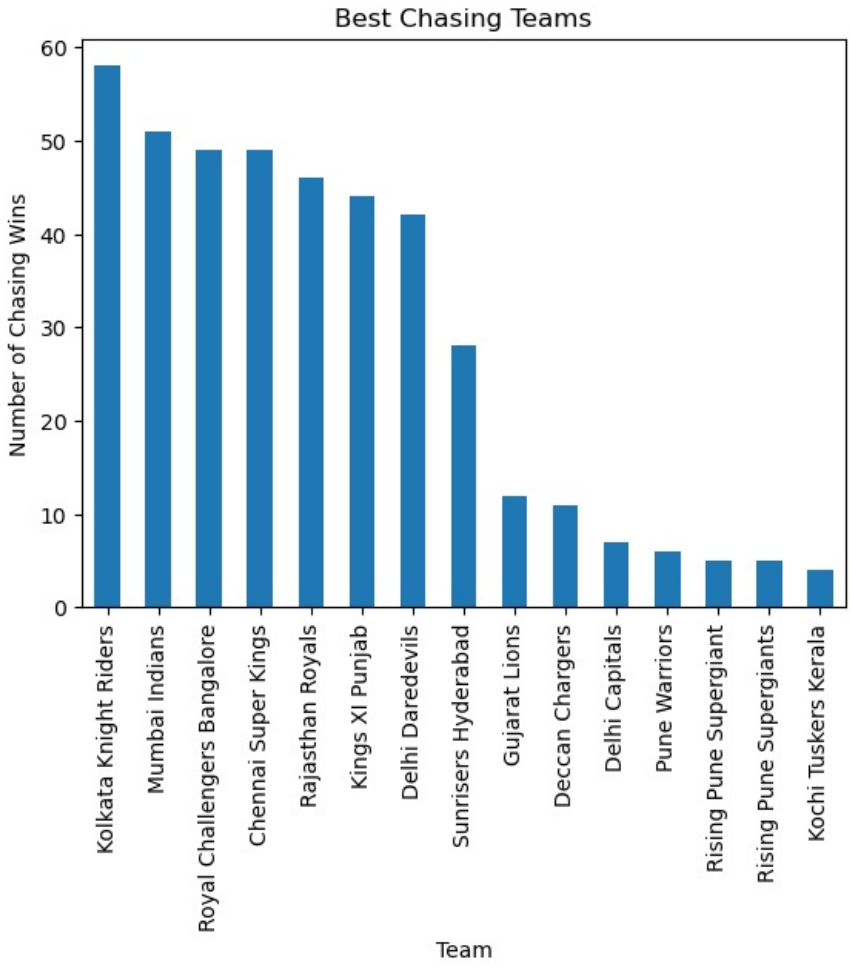
Key Observations

- Chasing wins dominate: Teams have won slightly more matches while chasing (just above 400) compared to defending (just below 350).
- Modern trend confirmed: This aligns with captains' preference to field first (from the earlier chart), showing that chasing is statistically more successful.

### xvi) Best defending and chasing teams.

In [29]:
```
import pandas as pd
import matplotlib.pyplot as plt

matches = pd.read_csv("matches.csv")

defend_wins = matches[matches["winner"] == matches["team1"]]["winner"].value_counts()

chase_wins = matches[matches["winner"] == matches["team2"]]["winner"].value_counts()

plt.figure()
defend_wins.plot(kind="bar")
plt.title("Best Defending Teams")
plt.xlabel("Team")
plt.ylabel("Number of Defending Wins")
plt.xticks(rotation=90)
plt.show()

plt.figure()
chase_wins.plot(kind="bar")
plt.title("Best Chasing Teams")
plt.xlabel("Team")
plt.ylabel("Number of Chasing Wins")
plt.xticks(rotation=90)
plt.show()
```




## Insights

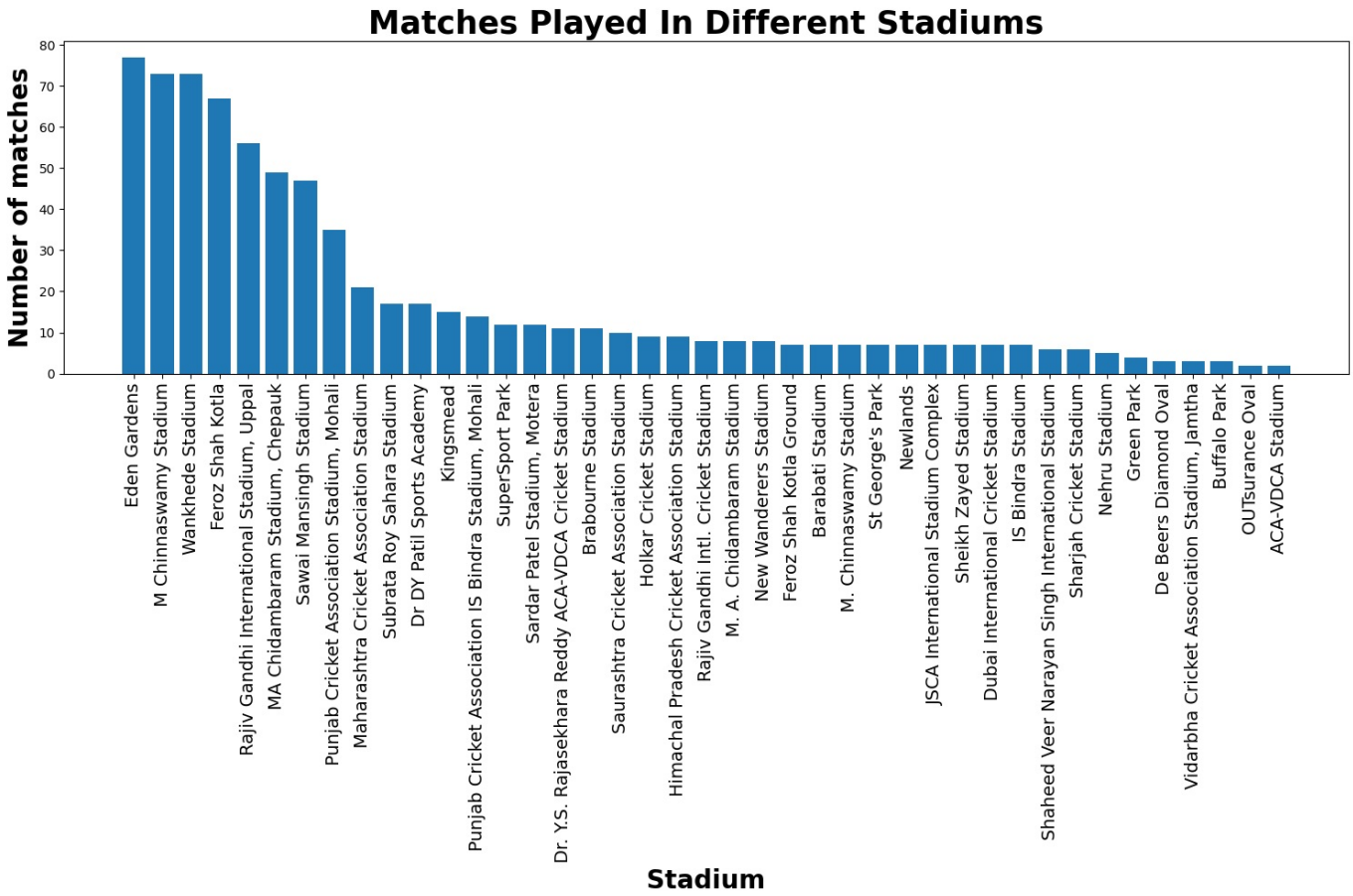
Key Observations

- Defending specialists:
- Mumbai Indians lead with ~57 defending wins, showing their strong bowling depth and ability to protect totals.

- Chennai Super Kings follow closely (~50), reflecting their disciplined bowling and tactical captaincy.
- Teams like Kings XI Punjab and Sunrisers Hyderabad also show notable defending strength, often relying on bowlers in home conditions.
- Chasing specialists:
- Kolkata Knight Riders top the chasing chart (~58 wins), highlighting their aggressive batting approach and ability to pace innings.
- Royal Challengers Bangalore (~48) and Chennai Super Kings (~45) also excel at chasing, thanks to strong batting lineups.
- Rajasthan Royals and Delhi Daredevils show consistent chasing success, often leveraging flexible batting orders. Strategic Insights
- Mumbai Indians are balanced: They appear near the top in both defending and chasing, showing adaptability across conditions.
- KKR tilt towards chasing: Their dominance in chasing suggests a batting-heavy strategy, aligning with modern trends favoring second innings.
- CSK's dual strength: High numbers in both defending and chasing reflect their tactical versatility under MS Dhoni's leadership.
- Smaller teams struggle: Short-lived franchises (Kochi Tuskers, Rising Pune Supergiants, Gujarat Lions) have very few wins in either category, due to limited seasons. Connecting Across All Charts
- Toss winners gain only a slight edge.
- Captains overwhelmingly choose to field, reflecting belief in chasing.
- Chasing statistically produces more wins overall.
- This chart confirms that team identity matters: some teams thrive defending (MI, CSK), while others thrive chasing (KKR, RCB).
- Success is not just about toss luck—it's about team composition, captaincy, and adaptability. I can now create a comparative matrix showing which teams are stronger in defending vs. chasing, so you can quickly revise team strategies side by side. Would you like me to build that structured comparison?

xvii) No. of matches played in different stadiums.

In [32]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("matches.csv")

matches_count = df['venue'].value_counts()

plt.figure(figsize=(15,10))
plt.bar(matches_count.index, matches_count.values)

plt.xlabel("Stadium",fontweight='bold',fontsize = 20)
plt.ylabel("Number of matches",fontweight='bold',fontsize = 20)
plt.title("Matches Played In Different Stadiums",fontweight='bold',fontsize = 25)

plt.xticks(rotation=90,fontsize = 14)
plt.tight_layout()
plt.show()
```



## Insights

Key Observations

- Eden Gardens tops the list: It has hosted the highest number of matches, reflecting its historic importance and large capacity.
- Other major venues: M. Chinnaswamy Stadium (Bangalore) and Wankhede Stadium (Mumbai) follow closely, both being iconic IPL venues with strong home-team associations.
- Regional spread: The chart shows matches distributed across many stadiums, highlighting the IPL's effort to expand cricket's reach across India and even internationally (Dubai, Sharjah).
- Short-lived venues: Some stadiums (like Kochi, Pune, or Buffalo Park in South Africa) hosted only a handful of matches, often due to temporary franchise participation or special circumstances